# Lista de exercícios – Recursividade

**Orientações para resolução da lista**

- Esta lista trabalha exclusivamente o conteúdo de **recursividade (fundamentos)**: pilha de chamadas, caso base, caso recursivo, o que acontece na ida e na volta das chamadas, comandos antes e depois da chamada recursiva e funções com mais de um caso base.
- Todas as questões são de **construção de código**. Em todas elas, a solução deve ser recursiva: **não utilize estruturas de repetição** (`for` ou `while`) dentro das funções recursivas.
- Antes de escrever o código, responda às três perguntas da aula: qual é o caso base? O problema diminui a cada chamada? O que acontece na ida e o que acontece na volta?
- Escreva primeiro a definição recursiva em papel, como foi feito com o fatorial, e só depois transforme cada linha da definição em uma parte do código.
- Para conferir sua solução, desenhe a pilha de chamadas para uma entrada pequena e acompanhe os valores devolvidos na volta.
- Lembre-se de que, sem um caso base alcançável, a pilha cresce até estourar (`RecursionError`). Verifique se toda chamada recursiva se aproxima do caso base.
- Desenvolva as soluções individualmente. O objetivo da lista é consolidar seu aprendizado e desenvolver autonomia na construção de programas.

## Exercícios

**1.** Crie uma função recursiva chamada `imprimir_decrescente(n)` que receba um número inteiro positivo e imprima os números de `n` até 1, um por linha.

Por exemplo, `imprimir_decrescente(4)` deve exibir:

```
4
3
2
1
```

No programa principal, leia o valor de `n` e chame a função. Em seguida, responda em um comentário no código: o que mudaria na saída se o comando `print` fosse colocado depois da chamada recursiva?

**2.** Crie uma função recursiva chamada `soma_pares(n)` que receba um número inteiro não negativo e devolva a soma de todos os números pares de 0 até `n`.

Por exemplo:

```
soma_pares(0)  -> 0
soma_pares(7)  -> 12
soma_pares(10) -> 30
```

No segundo caso, a soma é $6 + 4 + 2 + 0 = 12$. Observe que `n` pode ser ímpar: nesse caso, ele não participa da soma, mas a recursão deve continuar normalmente.

**3.** Crie uma função recursiva chamada `soma_digitos(n)` que receba um número inteiro não negativo e devolva a soma de seus algarismos.

Por exemplo:

```
soma_digitos(7)    -> 7
soma_digitos(5027) -> 14
soma_digitos(999)  -> 27
```

Utilize o resto da divisão por 10 para obter o último algarismo e a divisão inteira por 10 para obter o restante do número. Não converta o número para string.

**4.** Crie uma função recursiva chamada `produto_intervalo(a, b)` que receba dois números inteiros, com `a` menor ou igual a `b`, e devolva o produto de todos os inteiros de `a` até `b`.

Por exemplo:

```
produto_intervalo(3, 6) -> 360
produto_intervalo(5, 5) -> 5
produto_intervalo(1, 5) -> 120
```

No primeiro caso, o resultado é $3 \times 4 \times 5 \times 6 = 360$. Identifique com cuidado qual dos dois parâmetros deve mudar a cada chamada e qual é a situação que tem resposta imediata.

**5.** Crie uma função recursiva chamada `contar_caractere(texto, letra)` que receba uma string e um caractere e devolva quantas vezes esse caractere aparece no texto.

Por exemplo:

```
contar_caractere("banana", "a")    -> 3
contar_caractere("recursao", "r")  -> 2
contar_caractere("python", "z")    -> 0
contar_caractere("", "a")          -> 0
```

A cada chamada, analise o primeiro caractere do texto e continue o processamento com `texto[1:]`, que é o texto sem o primeiro caractere. Não utilize o método `count()`.

**6.** Crie uma função recursiva chamada `ampulheta(n)` que receba um número inteiro positivo e imprima uma figura formada por linhas de asteriscos. A figura começa com uma linha de `n` asteriscos, diminui um asterisco por linha até chegar a 1 e, em seguida, volta a crescer até `n`.

Por exemplo, `ampulheta(3)` deve exibir:

```
***
**
*
*
**
***
```

A função deve possuir uma única chamada recursiva. Pense no que deve ser impresso **antes** dessa chamada, na ida, e no que deve ser impresso **depois** dela, na volta. Lembre-se de que `"*" * 3` produz a string `"***"`.

**7.** Uma palavra é um palíndromo quando pode ser lida da mesma forma da esquerda para a direita e da direita para a esquerda, como *radar* e *arara*.

Crie uma função recursiva chamada `eh_palindromo(texto, inicio, fim)` que receba uma string e duas posições e devolva `True` se o trecho entre essas posições for um palíndromo, ou `False` caso contrário.

A chamada inicial deverá ser realizada da seguinte forma:

```
eh_palindromo(texto, 0, len(texto) - 1)
```

Por exemplo:

```
eh_palindromo("radar", 0, 4) -> True
eh_palindromo("casar", 0, 4) -> False
eh_palindromo("osso", 0, 3)  -> True
eh_palindromo("a", 0, 0)     -> True
```

A cada chamada, compare os caracteres das duas extremidades. Se forem diferentes, a resposta é imediata. Se forem iguais, continue a verificação com as posições seguintes, em direção ao centro. Quando as posições se encontrarem ou se cruzarem, não há mais nada a comparar.

**8.** Crie uma função recursiva chamada `maior_digito(n)` que receba um número inteiro não negativo e devolva o maior algarismo presente nele.

Por exemplo:

```
maior_digito(5027) -> 7
maior_digito(1111) -> 1
maior_digito(8)    -> 8
maior_digito(3941) -> 9
```

Quando o número possui um único algarismo, ele próprio é a resposta. Caso contrário, descubra recursivamente o maior algarismo do restante do número e compare-o com o último algarismo. Não converta o número para string e não utilize a função `max()`.

**9.** Uma pessoa deseja subir uma escada com `n` degraus. A cada passo, ela pode subir **1 degrau** ou **2 degraus** de uma vez.

Para uma escada com 3 degraus, por exemplo, existem 3 maneiras diferentes de subir: $1+1+1$, $1+2$ e $2+1$.

Crie uma função recursiva chamada `maneiras(n)` que devolva a quantidade de maneiras diferentes de subir uma escada com `n` degraus.

Por exemplo:

```
maneiras(1) -> 1
maneiras(2) -> 2
maneiras(3) -> 3
maneiras(4) -> 5
maneiras(6) -> 13
```

Para encontrar a definição recursiva, pense no **último passo**: ou a pessoa chegou ao topo subindo 1 degrau, vindo do degrau `n - 1`, ou subindo 2 degraus, vindo do degrau `n - 2`. Assim como na sequência de Fibonacci, serão necessários dois casos base e duas chamadas recursivas.

No programa principal, exiba o resultado da função para todas as escadas de 1 a 10 degraus.

**10.** Uma pessoa decide guardar dinheiro em uma aplicação financeira. No final de cada mês, o saldo acumulado até então recebe os juros do período e, logo em seguida, um novo depósito de valor fixo é realizado.

Considere, por exemplo, depósitos de R$ 100,00 e juros de 10% ao mês:

- ao final do mês 1, o saldo é 100,00 (apenas o primeiro depósito);
- ao final do mês 2, o saldo é $100{,}00 \times 1{,}10 + 100{,}00 = 210{,}00$;
- ao final do mês 3, o saldo é $210{,}00 \times 1{,}10 + 100{,}00 = 331{,}00$.

Antes do primeiro depósito, o saldo é zero.

Crie uma função recursiva chamada `saldo(meses, deposito, taxa)` que devolva o saldo acumulado ao final da quantidade de meses informada. A taxa deve ser recebida na forma decimal (por exemplo, `0.10` para 10%).

Por exemplo:

```
saldo(0, 100, 0.10) -> 0
saldo(1, 100, 0.10) -> 100.0
saldo(3, 100, 0.10) -> 331.0
```

Em seguida, crie uma segunda função recursiva, com a assinatura abaixo, que devolva quantos meses ainda são necessários para que o saldo alcance ou ultrapasse a meta, considerando que o parâmetro `acumulado` representa o saldo já existente. A chamada inicial deverá utilizar `acumulado` igual a zero.

```
meses_para_meta(meta, deposito, taxa, acumulado)
```

Por exemplo:

```
meses_para_meta(300, 100, 0.10, 0)  -> 3
meses_para_meta(100, 100, 0.10, 0)  -> 1
meses_para_meta(1000, 100, 0.10, 0) -> 8
```

Nessa segunda função, o problema diminui porque o saldo acumulado se aproxima da meta a cada chamada.

No programa principal, leia o valor do depósito mensal, a taxa de juros e a meta desejada. Exiba a quantidade de meses necessária e o saldo ao final desse período, calculado pela primeira função.

Não utilize estruturas de repetição em nenhuma das duas funções.